

Laboratory 3 — Transforming Textual and Image Data into a Machine-Understandable Format

Course: Introduction to Artificial Intelligence (AI-101L) · **Instructor:** Dr. Muhammad Awais
Duration: 3 hours · **Block:** Phase A — Environment and Data · **Mapped CLO:** CLO-2

Learning Objectives

On completion of this laboratory you will be able to:

1. Explain why raw text and raw images cannot be fed directly to a traditional machine learning algorithm.
2. Apply the **Bag-of-Words** model with `CountVectorizer` to convert a corpus into a numerical document-term matrix.
3. Convert an image into a numerical array of pixel values with **Pillow** and **NumPy**.
4. Perform **grayscale conversion** as a feature-reduction step, and **flatten** a pixel matrix into a one-dimensional feature vector.
5. State precisely **what information each transformation discards**.

Background

Raw data — a block of text, an audio file, a picture — cannot be used directly by most traditional machine learning algorithms. Those algorithms require a **numerical, vector-based input**. *Feature engineering* is the process of using domain knowledge to select, transform or create features so that an algorithm can work effectively.

Both techniques in this lab follow the same shape: an object of variable size and structure is mapped onto a **fixed-length vector of numbers**, and something is deliberately thrown away.

```
Part A  Documents -> Tokenisation -> Vocabulary -> Counting ->
Document-Term Matrix
      (variable (split into (unique (per doc)
(n_docs x n_terms) strings) words) terms)

Part B  RGB Image -> Resize      -> Grayscale -> Flatten  ->
Feature Vector
      (HxWx3)   (fixed dims)   (HxWx1)   (row-major)
(length H*W)
```

both paths end in a fixed-length numeric vector

Instructor note. No data files are required. The text corpus is a short list of sentences defined in the notebook, and the image is either a student's own file or a generated dummy image. The lab therefore runs on any machine, offline.

Setup and prerequisites

Library	Purpose	Installation
scikit-learn	CountVectorizer for text feature extraction	<code>pip install scikit-learn</code>
Pillow (PIL)	Image loading and manipulation	<code>pip install Pillow</code>
NumPy	Numerical array operations	<code>pip install numpy</code>

```
In [1]: !pip install scikit-learn Pillow numpy pandas matplotlib
```

```
Requirement already satisfied: scikit-learn in C:\Program Files\Python313\Lib\site-packages (1.9.0)
Requirement already satisfied: Pillow in C:\Program Files\Python313\Lib\site-packages (11.3.0)
Requirement already satisfied: numpy in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (2.4.6)
Requirement already satisfied: pandas in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (3.0.3)
Requirement already satisfied: matplotlib in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (3.10.9)
Requirement already satisfied: scipy>=1.10.0 in C:\Program Files\Python313\Lib\site-packages (from scikit-learn) (1.18.0)
Requirement already satisfied: joblib>=1.4.0 in C:\Program Files\Python313\Lib\site-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: narwhals>=2.0.1 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from scikit-learn) (2.22.1)
Requirement already satisfied: threadpoolctl>=3.5.0 in C:\Program Files\Python313\Lib\site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: tzdata in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from pandas) (2026.2)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pyparsing>=3 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in C:\Users\HP\AppData\Roaming\Python\Python313\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

```
[notice] A new release of pip is available: 26.2 -> 26.2.1
```

```
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Part A — Feature engineering for text data

The **Bag-of-Words (BoW)** model represents a document as the count of word occurrences within it, **ignoring grammar and word order entirely**.

Task 3.1 — Vectorising text with CountVectorizer

Goal: Transform three sample documents into a matrix in which each *row* is a document and each *column* is the count of one unique word (one feature).

```
In [2]: import pandas as pd
        from sklearn.feature_extraction.text import CountVectorizer

        # 1. Define Sample Text Data (our "corpus")
        documents = [
            "The sun is shining today.",
            "The weather is good, the sun is great.",
            "A sunny day is a wonderful day."
        ]

        print("--- Raw Text Documents ---")
        for i, doc in enumerate(documents):
            print(f"Doc {i+1}: {doc}")
```

```
--- Raw Text Documents ---
Doc 1: The sun is shining today.
Doc 2: The weather is good, the sun is great.
Doc 3: A sunny day is a wonderful day.
```

```
In [3]: # 2. Apply Feature Engineering: CountVectorizer (Bag-of-Words)
        # CountVectorizer performs tokenization AND vocabulary building in one step.
        vectorizer = CountVectorizer()

        # fit_transform learns the vocabulary and converts the documents into feature vectors
        X_text = vectorizer.fit_transform(documents)

        # 3. Analyze the Results
        feature_names = vectorizer.get_feature_names_out()
        text_matrix = X_text.toarray()

        print("--- Bag-of-Words (BoW) Transformation ---")
        print(f"Vocabulary (Features): {feature_names}")
        print(f"Shape of Feature Matrix: {text_matrix.shape} (documents x unique terms)")
```

```
--- Bag-of-Words (BoW) Transformation ---
Vocabulary (Features): ['day' 'good' 'great' 'is' 'shining' 'sun' 'sunny' 'the'
                        'today' 'weather'
                        'wonderful']
Shape of Feature Matrix: (3, 11) (documents x unique terms)
```

```
In [4]: # A DataFrame view, purely so the matrix is readable
df_text = pd.DataFrame(text_matrix, columns=feature_names,
                        index=[f"Doc {i+1}" for i in range(len(documents))])

print("Bow Numerical Feature Matrix (Machine Understandable Format):\n")
display(df_text)
```

Bow Numerical Feature Matrix (Machine Understandable Format):

	day	good	great	is	shining	sun	sunny	the	today	weather	wonderful
Doc 1	0	0	0	1	1	1	0	1	1	0	0
Doc 2	0	1	1	2	0	1	0	2	0	1	0
Doc 3	2	0	0	1	0	0	1	0	0	0	1

Expected output

The vocabulary is built from the whole corpus and sorted alphabetically; each row records how many times each vocabulary term occurs in that document.

	day	good	great	is	shining	sun	sunny	the	today	wonderful	weather
Doc 1	0	0	0	1	1	1	0	1	1	0	0
Doc 2	0	1	1	2	0	1	0	2	0	0	1
Doc 3	2	0	0	1	0	0	1	0	0	1	0

Note the two silent decisions

`CountVectorizer` **lowercases** every token by default, so *The* and *the* collapse into one feature; and its default token pattern **discards single-character tokens**, so the "a" of Document 3 never enters the vocabulary. Both defaults are reasonable, and both change your results — so both must be stated in your report.

```
In [5]: # Prove the two silent decisions to yourself rather than taking them on trust.

print("Is 'a' in the vocabulary? ", 'a' in feature_names)
print("Is 'the' in the vocabulary?", 'the' in feature_names)
print()
print("Default token pattern :", vectorizer.token_pattern)
print("Lowercase enabled    :", vectorizer.lowercase)
print()
print("The token pattern requires 2+ word characters, which is why 'a' is dropped")
```

Is 'a' in the vocabulary? False
Is 'the' in the vocabulary? True

Default token pattern : (?u)\b\w+\b
Lowercase enabled : True

The token pattern requires 2+ word characters, which is why 'a' is dropped.

```
In [6]: # Sparsity: how much of this matrix is zero? This is the defining property of Bo
total_cells = text_matrix.size
zero_cells = (text_matrix == 0).sum()

print(f"Matrix cells : {total_cells}")
print(f"Zero cells   : {zero_cells}")
print(f"Sparsity     : {zero_cells / total_cells:.1%}")
print()
print("With a real corpus of 20,000 documents this figure exceeds 99%,")
print("which is why scikit-learn returns a SPARSE matrix, not a dense array.")
print("Type returned by fit_transform:", type(X_text))
```

Matrix cells : 33
Zero cells : 18
Sparsity : 54.5%

With a real corpus of 20,000 documents this figure exceeds 99%,
which is why scikit-learn returns a SPARSE matrix, not a dense array.
Type returned by fit_transform: <class 'scipy.sparse._csr.csr_matrix'>

Discussion questions (Part A)

Answer these in the Markdown cell below.

1. What does each **column** in the resulting matrix represent?
2. In this specific numerical matrix, what is the *machine-understandable format*?
3. How is this format an **oversimplification** of the original text data? (Consider what information has been lost.)

Your answers (Part A):

1. (double-click and write here)
 2. (...)
 3. (...)
-
-

Part B — Feature engineering for image data

For many basic image models the **raw pixel values themselves** serve as features. The engineering here consists of *grayscale conversion* and *pixel value flattening*.

Task 3.2 — Converting and flattening image pixels

Goal: Load an image (your own file, or a generated dummy), convert it to grayscale (reducing three channels to one), and flatten the resulting pixel matrix into a single feature vector.

```
In [7]: import numpy as np
        from PIL import Image
        import os

        # --- Configuration for image loading -----
        # Set to None to use a dummy image, or to a path string for your own image.
        # e.g. 'my_image.png' or 'C:/Users/You/Pictures/photo.jpg'
        image_file_path = None

        # 1. Load Image or Create a Dummy Image
        original_image = None
        if image_file_path and os.path.exists(image_file_path):
            try:
                original_image = Image.open(image_file_path)
                print(f"--- Loaded Image from File: {image_file_path} ---")
            except Exception as e:
                print(f"Error loading image from {image_file_path}: {e}")
                dummy = np.random.randint(0, 256, size=(100, 100, 3), dtype=np.uint8)
                original_image = Image.fromarray(dummy, 'RGB')
                print("--- Created Dummy Image (100x100 RGB) ---")
        else:
            print("No valid image file path provided or file not found.")
            dummy = np.random.randint(0, 256, size=(100, 100, 3), dtype=np.uint8)
            original_image = Image.fromarray(dummy, 'RGB')
            print("--- Created Dummy Image (100x100 RGB) ---")

        # Ensure a consistent 3-channel representation
        original_image = original_image.convert('RGB')
        original_image_array = np.array(original_image)

        h, w, c = original_image_array.shape
        print(f"Original Image Size (H, W, Channels): {original_image_array.shape}")
        print(f"Total Features (Pixels) in RGB: {h * w * c}")
```

No valid image file path provided or file not found.

--- Created Dummy Image (100x100 RGB) ---

Original Image Size (H, W, Channels): (100, 100, 3)

Total Features (Pixels) in RGB: 30000

C:\Users\HP\AppData\Local\Temp\ipykernel_14220\88318890.py:24: DeprecationWarning: 'mode' parameter is deprecated and will be removed in Pillow 13 (2026-10-15)
original_image = Image.fromarray(dummy, 'RGB')

```
In [8]: # 2. Feature Engineering step 1: Grayscale Conversion ('L' mode = Luminance)
        grayscale_image = original_image.convert('L')
        grayscale_array = np.array(grayscale_image)

        print("--- Grayscale Conversion (Feature Reduction) ---")
        print(f"Grayscale Image Size (H, W): {grayscale_array.shape}")
        print(f"Total Features (Pixels) after Grayscale: {grayscale_array.size}")
        print()
        print(f"Reduction factor: {(h * w * c) / grayscale_array.size:.0f}x fewer features")
```

```

--- Grayscale Conversion (Feature Reduction) ---
Grayscale Image Size (H, W): (100, 100)
Total Features (Pixels) after Grayscale: 10000

```

Reduction factor: 3x fewer features

```

In [9]: # 3. Feature Engineering step 2: Pixel Flattening (2D matrix -> 1D vector)
        flattened_features = grayscale_array.flatten()

        print("--- Pixel Flattening (Vectorization) ---")
        print(f"Flattened Feature Vector Shape: {flattened_features.shape}")
        print()
        print("First 10 Features (Pixel Values) in Machine Understandable Format:")
        print(flattened_features[:10])
        print()
        print(f"Data Type of Final Features: {flattened_features.dtype}")

```

```

--- Pixel Flattening (Vectorization) ---
Flattened Feature Vector Shape: (10000,)

```

```

First 10 Features (Pixel Values) in Machine Understandable Format:
[ 82  63 172 204  88 149 146  99 125 142]

```

Data Type of Final Features: uint8

```

In [10]: # Show all three stages side by side so the loss of information is visible.
        import matplotlib.pyplot as plt

        fig, axes = plt.subplots(1, 3, figsize=(14, 4))

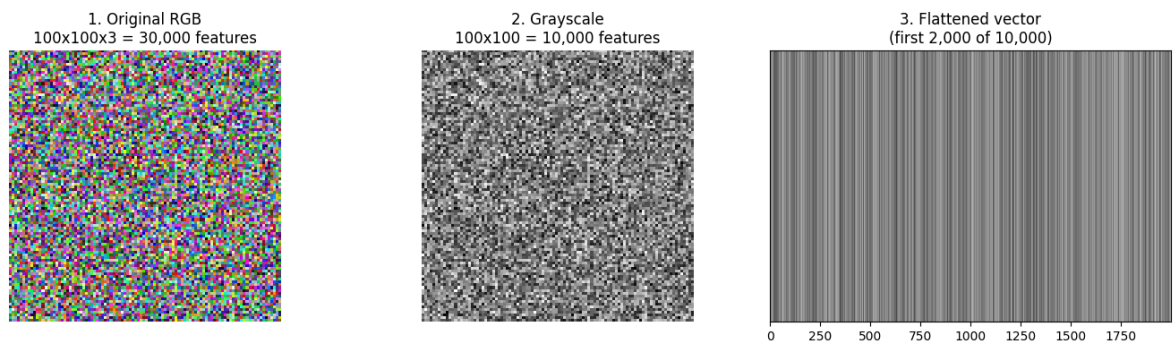
        axes[0].imshow(original_image)
        axes[0].set_title(f"1. Original RGB\n{h}x{w}x{c} = {h*w*c:,} features")
        axes[0].axis('off')

        axes[1].imshow(grayscale_image, cmap='gray')
        axes[1].set_title(f"2. Grayscale\n{h}x{w} = {h*w:,} features")
        axes[1].axis('off')

        # A 1-D vector drawn as a single row of pixels
        axes[2].imshow(flattened_features[:2000].reshape(1, -1), cmap='gray', aspect='auto')
        axes[2].set_title(f"3. Flattened vector\n(first 2,000 of {flattened_features.size:,} features)")
        axes[2].set_yticks([])

        plt.tight_layout()
        plt.show()

```



Expected output (dummy image)

```
--- Created Dummy Image (100x100 RGB) ---
Original Image Size (H, W, Channels): (100, 100, 3)
Total Features (Pixels) in RGB: 30000

--- Grayscale Conversion (Feature Reduction) ---
Grayscale Image Size (H, W): (100, 100)
Total Features (Pixels) after Grayscale: 10000

--- Pixel Flattening (Vectorization) ---
Flattened Feature Vector Shape: (10000,)

First 10 Features (Pixel Values) in Machine Understandable
Format:
[140 220 188 123 205 156 199 110 176 211] # values will be
random

Data Type of Final Features: uint8
```

Grayscale is a three-to-one feature reduction

A 100×100 RGB image carries $100 \times 100 \times 3 = 30,000$ features; after grayscale conversion it carries 10,000. **Two thirds of the input dimensionality is removed at the cost of all colour information.** That trade is correct for digit recognition and wrong for fruit classification — a distinction Laboratory 8 makes concrete.

Discussion questions (Part B)

1. Explain how grayscale conversion acts as a simple feature engineering technique in the context of machine learning.
2. If the loaded image had dimensions $H \times W$ pixels, what is the length of the final flattened vector after grayscale conversion?
3. Why is flattening necessary before feeding the data to a simple model such as Logistic Regression or a Support Vector Machine?
4. What is the advantage of **resizing all images to a consistent size** (e.g. 64×64) before grayscale conversion and flattening, when the model must process many different images?

Your answers (Part B):

1. *(double-click and write here)*
 2. (...)
 3. (...)
 4. (...)
-
-

In-Lab Exercises

Exercise 1. Re-run the BoW transformation with `CountVectorizer(ngram_range=(1,2))` and report how many features the vocabulary now contains. Explain the growth.

Exercise 2. Add a fourth document that repeats the word *sun* five times. Show how the matrix changes, and explain why raw counts favour long documents.

Exercise 3. Resize a real photograph to 64×64 , then to 32×32 , and display both alongside the original. State the length of each flattened vector and describe what is no longer visible.

Exercise 4. Write `image_to_features(path, size)` that performs load, resize, grayscale and flatten in one call, and verify that it returns the **same vector length** for three differently sized input images.

```
In [11]: # --- Exercise 1: unigrams + bigrams -----
```

```
vec_1gram = CountVectorizer(ngram_range=(1, 1))
vec_2gram = CountVectorizer(ngram_range=(1, 2))

X1 = vec_1gram.fit_transform(documents)
X2 = vec_2gram.fit_transform(documents)

print(f"ngram_range=(1,1) -> {X1.shape[1]:3d} features")
print(f"ngram_range=(1,2) -> {X2.shape[1]:3d} features")
print(f"Growth factor      : {X2.shape[1] / X1.shape[1]:.1f}x")
print()
print("New bigram features introduced:")
print([f for f in vec_2gram.get_feature_names_out() if ' ' in f])
```

```
ngram_range=(1,1) -> 11 features
ngram_range=(1,2) -> 24 features
Growth factor      : 2.2x
```

New bigram features introduced:

```
['day is', 'good the', 'is good', 'is great', 'is shining', 'is wonderful', 'shin
ing today', 'sun is', 'sunny day', 'the sun', 'the weather', 'weather is', 'wonderful day']
```

```
In [12]: # --- Exercise 2: raw counts favour Long documents -----
```

```
documents_4 = documents + ["sun sun sun sun sun and more sun today"]

vec = CountVectorizer()
M = vec.fit_transform(documents_4).toarray()
df4 = pd.DataFrame(M, columns=vec.get_feature_names_out(),
                   index=[f"Doc {i+1}" for i in range(len(documents_4))])
display(df4)

print("Total token count per document (row sums):")
print(df4.sum(axis=1).to_string())
print()
print("Doc 4 dominates the 'sun' column purely by repetition.")
print("This is exactly the problem TF-IDF (Lab 02) was designed to correct.")
```

	and	day	good	great	is	more	shining	sun	sunny	the	today	weather	wond
Doc 1	0	0	0	0	1	0	1	1	0	1	1	0	
Doc 2	0	0	1	1	2	0	0	1	0	2	0	1	
Doc 3	0	2	0	0	1	0	0	0	1	0	0	0	
Doc 4	1	0	0	0	0	1	0	6	0	0	1	0	

Total token count per document (row sums):

```
Doc 1    5
Doc 2    8
Doc 3    5
Doc 4    9
```

Doc 4 dominates the 'sun' column purely by repetition.

This is exactly the problem TF-IDF (Lab 02) was designed to correct.

```
In [13]: # --- Exercise 4: a reusable image -> feature-vector function -----

def image_to_features(path, size=(64, 64)):
    """Load, resize, grayscale and flatten an image into a 1-D feature vector.

    Parameters
    -----
    path : str
        Path to an image file.
    size : tuple[int, int]
        Target (width, height). Every image is forced to this size so that
        every returned vector has the SAME length -- which is what a model
        with a fixed input layer requires.

    Returns
    -----
    numpy.ndarray of shape (size[0] * size[1],), dtype uint8
    """
    img = Image.open(path).convert("RGB") # normalise channel count first
    img = img.resize(size)                # then normalise dimensions
    img = img.convert("L")                 # then reduce to one channel
    return np.array(img).flatten()         # finally vectorise

# Verify the guarantee on three differently sized synthetic images.
os.makedirs("test_images", exist_ok=True)
for i, dims in enumerate([(120, 80), (300, 300), (64, 200)], start=1):
    arr = np.random.randint(0, 256, size=(dims[1], dims[0], 3), dtype=np.uint8)
    Image.fromarray(arr, "RGB").save(f"test_images/img{i}.png")

for i, dims in enumerate([(120, 80), (300, 300), (64, 200)], start=1):
    v = image_to_features(f"test_images/img{i}.png", size=(64, 64))
    print(f"img{i}.png original {dims[0]}x{dims[1]} -> vector length {v.shape}")

print("\nAll three vectors have identical length -- the model can accept any of
img1.png original 120x80 -> vector length 4096
```

```
img2.png original 300x300 -> vector length 4096
img3.png original 64x200 -> vector length 4096
```

All three vectors have identical length -- the model can accept any of them.

```
C:\Users\HP\AppData\Local\Temp\ipykernel_14220\3926732397.py:29: DeprecationWarning: 'mode' parameter is deprecated and will be removed in Pillow 13 (2026-10-15)
  Image.fromarray(arr, "RGB").save(f"test_images/img{i}.png")
```

Home Assignment

Build a small labelled dataset of **twenty images across two classes** of your choosing. Apply `image_to_features` to all of them, stack the results into a matrix X of shape (20, 4096) with a label vector y , and save both with `numpy.savez`.

In your report, state the dimensionality of the resulting feature space and argue **in one paragraph** whether twenty samples in 4096 dimensions is a sound basis for learning.

```
In [14]: # --- Home assignment scaffold -----
# Put your images in data/class_a/ and data/class_b/ then run this cell.

from glob import glob

CLASS_DIRS = {0: "data/class_a", 1: "data/class_b"}
SIZE = (64, 64)

X_list, y_list = [], []
for label, folder in CLASS_DIRS.items():
    paths = sorted(glob(os.path.join(folder, "*")))
    for p in paths:
        try:
            X_list.append(image_to_features(p, size=SIZE))
            y_list.append(label)
        except Exception as e:
            print(f"[SKIP] {p} -> {e}")

if X_list:
    X = np.vstack(X_list)
    y = np.array(y_list)
    np.savez("dataset.npz", X=X, y=y)

    print(f"X shape: {X.shape}   y shape: {y.shape}")
    print(f"Samples: {X.shape[0]}   Features per sample: {X.shape[1]}")
    print(f"Samples per feature: {X.shape[0] / X.shape[1]:.4f}")
    print()
    print("Saved dataset.npz")
else:
    print("No images found. Create data/class_a and data/class_b and add images.")
```

No images found. Create data/class_a and data/class_b and add images.

Your paragraph on 20 samples in 4096 dimensions:

(double-click and write here — consider the curse of dimensionality, and what Laboratory 6 on dimensionality reduction, or a CNN in Laboratory 8, would do differently)

Deliverables

File	Contents
lab03/text_features.ipynb	Part A, executed
lab03/image_features.ipynb	Part B, executed
lab03/image_utils.py	The <code>image_to_features</code> function
lab03/dataset.npz	The 20-image labelled dataset
lab03/report.md	Answers to all seven discussion questions

Assessment Rubric

Criterion	Weight	Excellent performance
Correctness of implementation	35 %	Both transformations produce the specified shapes
Methodological soundness	25 %	Fixed vector length guaranteed; defaults stated explicitly
Analysis and interpretation	20 %	What each transformation <i>discards</i> is named precisely
Code quality and reproducibility	10 %	<code>image_to_features</code> is reusable and documented
Report and demonstration	10 %	All seven discussion questions answered

Next laboratory: Lab 04 — Introduction to Scikit-learn and Traditional Machine Learning.